ate

Assembly Language (RISC)

Architecture design Principles:

RISC → Reduced Instruction Set Computer.
R I S C

4 design principles:

1) Simplicity favours Regularity.
2) Make the common case fast
3) Smaller is faster
4) Good designs demand good compromises.

Design Principle 1

Simplicity favors regularity

- Consistent instruction format
- Same number of operands (two sources and one destination)
- Easier to encode and handle in hardware

BHATTI

CamScanner

Design Principle 2

Make the common case fast

- RISC-V includes only simple, commonly used instructions
- Hardware to decode and execute instructions can be simple, small, and fast
- More complex instructions (that are less common) performed using multiple simple instructions
- RISC-V is a ***reduced instruction set computer*** (RISC), with a small number of simple instructions
- Other architectures, such as Intel's x86, are ***complex instruction set computers*** (CISC)

Design Principle 3

Smaller is Faster

- RISC-V includes only a small number of registers

Design Principle 4

Good design demands good compromises

- Multiple instruction formats allow flexibility
  - `add`, `sub`: use 3 register operands
  - `lw`, `sw`, `addi`: use 2 register operands and a constant
- Number of instruction formats kept small
  - to adhere to design principles 1 and 3 (simplicity favors regularity and smaller is faster).

CamScanner

day/date

RISC-V Registers:

There are 3 types of operands.

1) Registers
2) memory
3) Constants / immediates.

Risc-V has 32-registers that are all of 32-bits.

Registers are faster than memory & Almost all instruction in risc-V use registers only.

-Next page shows all 32 registers with their functionalities.

BHATTI

CamScanner

ay/date

Name	Register Number	Usage
zero	x0	constant value 0.
ra	x1	return address.
sp	x2	Stack pointer
gp	x3	global pointer
tp	x4	thread pointer
t0, t1, t2	x5, x6, x7	Temporaries
s0/fp	x8	saved register / frame pointer
s1	x9	saved register
a0, a1	x10, x11	Function argument / return values.
a2, a3 ... a7	x12, x13 .... x17	Function arguments.
s2, s3 ... s11	x18, x19 ... x27	Saved registers.
t3, t4, t5, t6	x28, x29, x30, x31	Temporaries.

BHATTI

CamScanner

y/date

1) Zero register: (x_0)

- It's value is always 0.
- we cannot change its value

2) Temporaries (t_0 --- t_6)

- 7 registers
- used to store temporary data between operations.

x_5, x_6, x_7 , $x_{28}, x_{29}, x_{30}, x_{31}$.

3) Saved registers (s_0 --- s_{11}).

Used to load memory or save back to memory.

- 12 registers.
- So can be used as a frame pointer as well.

BHATTI

CamScanner

y/date

4) Function arguments: ($a_0 - a_7$)

To pass function arguments.

- 8 registers

a_0 & a_1 can also be used for Return values.

$x_{10}, x_{11}, x_{12}, x_{13}, x_{14}, x_{15}, x_{16}, x_{17}$

Important:

1) To use a register we can use $x_0, x_1 \cdots x_{32}$ or its name e.g zero, a_0, s_1, t_2, sp.

2) x_0 cannot be changed.

3) x_1 & x_4 can be changed but can lead to program crashing.

4) Any register can be used for anything. We follow this naming convention to establish a social contract between compilers & Programmers.

BHATTI

CamScanner

/date

Basic Arthemetic Instructions:

RISC-V arthemetic instructions strictly require 3 operands:

mnemonic rd, rs1, rs2.

rd = rs1 ∓ rs2. ← means.

Add:

Add rd, rs1, rs2.

rd = rs1 + rs2.

BHATTI

CamScanner

day/date

Subtraction

Sub rd , rs1 , rs2

example:

Sub x5 , x6 , x7

x5 = x6 + x7.

Immediate Addition:

If we want to add a constant then we use:

addi rd , rs1 , immediate

e.g

addi x5 , x6 , 20

x5 = x6 + 20

BHATTI

CamScanner

/date

There is no "subi" in RISC-V, instead we use "addi" for that qual.

addi x5, x6, -5;

This is sign extended.

$x5 = x6 - 5$.

- Also the immediate has a max limit: This will be discussed later in generating constants.

Imp:

All these 3 instructions follow 3 operands rule., which keeps the design simple & decoding hardware simple & fast.

Subi also not exists because we can do same with addi thus subi is redundant & not in RISC-V.

BHATTI

CamScanner

date

Understanding Memory:

We cannot store all of our data in 32 registers, so we need to use memory for it.

In RISC-V:

Byte = 8 bits.

WORD = 4 bytes = 32 bits

RISC-V is byte addressable. & follows little endian

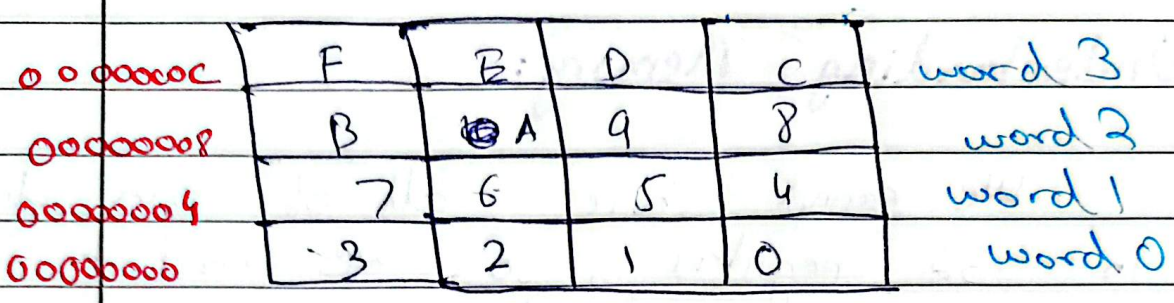
Byte-addressable:

In byte-addressable architecture, each distinct address points to 1 Byte (8 bits) of data.

BHATTI

CamScanner

day/date



every box represents one byte

Alignment restriction: Storing a 32-bit word at an address that isn't divisible by 4 (like 0x01 or 0x03) is called an unaligned access. RISC V requires aligned access for max speed.

BHATTI

CamScanner

day/date

Reading from memory:

for this we use load from memory:

lw destination register , offset (Base register)

lw loads 4 consecutive bytes from RAM starting at the (Offset + Base register) value

offset (base Register):

e.g 12 (x10) — register x10.

consider x10 has 0x110 in it

so

starting address = 12 + 0x110 = 0x11C

* always remember hexadecimal & decimal.

very imp.

addresses are in hexadecimal.

BHATTI

CamScanner

day/date

Saving to memory:

we use:

sw source register, offset (base register).

same as load word but now save word.

- works like load word as well. starting address (offset + base register)

* keep little endian in mind if you are seeing memory addresses.

* keep hexadecimal in mind for addressing.

BHATTI

CamScanner

day/date

Generating Constants:

In risc-v every standard instruction is fixed at 32 bits (4 bytes) wide.

so e.g. add r1, r2, r3

all this needs to fit in 32 bit.

now for immediates:

addi r1, r2, immediate

every register is 32 bit so 2^5

so 5 bits needed to address each register.

$32 - 2*5 = 22$ bits left.

after this the "addi" & other bits, like flags, signals also take bits leaving immediate with 12 bits.

So in immediate we can max get 12-bit signed constants. using addi.

BHATTI

day/date

But what if we want lets say 0x FEDC8765, now this needs 32 bits.

used to hexadecimal.

now for this a programmer needs to understand that he cannot fetch this from memory (because we haven't placed it there yet).
we can add numbers again & again to reach result but this takes huge amount of instructions.

So what to do?

How to generate the required constant?

so what we do is we divide our number (converted to hex before hand) into 2 parts.

0x FEDC8765
20 bits. 12 bits

BHATTI

CamScanner

day/date

* Remember to convert a decimal to hexa beforehand. (It makes division into 2 parts easier).

After this the programmer needs to load the upper bits of the register with the first 20 bits.

lui register , immediate

Load upper immediate.

This instruction has enough space to leave 20 bits for immediate

so

```
addi  s0 , x0 , 0 ; clearing register
lui   s0 , 0x FED C8
```

Now

s0 = 0X FE DC8 000

unite in to hexa decimal we use 0 x — — way in RISC-V

now we need to add the last 12 bits.

BHATTI

CamScanner

day/date

now just use the addi command to add the last bits

```
addi s0 , x0 , 0 ;
lui  s0 , 0xFEDC8
addi s0 , s0 , 0x765;
```

That is how to generate constants.

Now another problem:

In hardware 2 same size operands can only be added.
i.e adder in hardware is defined like 32-bit adder, both operands should be 32-bit

but in addi, our immediate is 12 bits.

BHATTI

day/date

So what addi does is it "sign extends" the 12 bit number.

e.g in last: 0x F12DC8000
+ 0x0 00000768
↑ +ve so sign extended.

But this leads to another problem:
If the last 12 bits are 3rd Byte only is a negative number itself e.g

0x865 extended becomes
0xFFFFF865
↑ -ve sign extended.

This would make it wrong!!

This would lead our answer to be one less in the 4th Bytes place.

BHATTI

CamScanner

day/date

e.g

move 0x12348A00 into x5.

```
addi  x5, x0, 0;
lui   x5, 0x12348
addi  x5, x5, 0xA00
```

Problem:

```
  0x12348000
+ 0xFFFFFA00
------------
  12347A00
```

① ← Last carry bit

wrong answer it should be 12348A00

This byte is one less.

So the solution to this:

we need to increase our upper 5 bits by one.

BHATTI

CamScanner

day/date

e.g

instead of lui x5, 0x12348

do lui x5, 0x12349.

But remember we only do this if our lower 12 bits are leading to a negative sign extension.

How to know lower 12 bits are leading to negative extension.

e.g 0 X 765

If 3rd byte number ≤ 7 then it is +ve because 7 = 0111 (0 last bit)

If 3rd byte ≥ 8 (1000) then it will be -ve sign extension (1 is when signalling -ve)

& we need to add 1 in upper 5 bits.

BHATTI

CamScanner

Clearing registers:

To clear registers, there are 2 ways that I know of currently:

add x5 , x0 , x0

x5 = 0 + 0 = 0 .

addi x5 , zero , 0;

x5 = 0 + 0 = 0 .

BHATTI

CamScanner

day/date

Logical & Shift Instructions:

AND:

AND rd , rs1 , rs2.

rd (register destination) = rs1 & rs2

registers only.

OR:

OR rd , rs1 , rs2.

rd = rs1 or rs2.

XOR:

XOR rd , rs1 , rs2.

rd = rs1 ⊕ rs2.

BHATTI

CamScanner

day/date

Imp:

The operands are only registers which all are 32 bits. So when you are actually solving them use 32 bits completely.

Immediates:

```
andi  rd , rs , immediate
```

```
ori  rd , rs , immediate
```

```
xori  rd , rs , immediate
```

again here immediate is max 12 bit & sign extended.

If we need to do with bigger number, first generate constant & then use and, or, xor between 2 registers.

BHATTI

CamScanner

day/date

Shift Instructions.

1) Shift left logical

`sll rd, rs1, rs2`

rd = rs1 << rs2.

rs1 — number; rs2 — amount of time to shift.

2) Shift right logical

`srl rd, rs1, rs2.`

rd = rs1 >> rs2.

3) Shift right arithmetic.

`sra rd, rs1, rs2.`

rd = rs1 >>> rs2.

This keeps sign same.

BHATTI

CamScanner

day/date

Immediate Shifts:

slli rd, rs1, immediate

srli rd, rs1, immediate

srai rd, rs1, immediate

immediate value should be in range: 0-31

BHATTI

CamScanner

/date

Multiplication :

we know that a 32 bit * 32 bit can lead to a max answer of 64 bit.

but we only have 32 bit registers.

so :

`mul rd , rs1 , rs2` — stores lower 32 bits in rd.

`mulh rd , rs1 , rs2` — stores upper 32 bits in rd.

rd = rs1 * rs2.

BHATTI

CamScanner

ay/date

Division:

32 bit division can lead to 32 bit quotient & 32 bit remainder.

`div rd , rs1 , rs2.` integer division

rd = rs1 / rs2.

`rem rd , rs1 , rs2` → always result gets sign of dividend (rs1).

rd = rs1 % rs2.

What if we divide by 0?

Typically this should give error but in RISC-V, it is handled differently

$$rd = \frac{rs1}{0} = -1 \ (0xFFFFFFFF).$$

rd = rs1 % 0 = rs1.

BHATTI

CamScanner

day/date

dividing by 0, makes quotient -1
& remainder the original divisor.

Comment in RISC-V:

for comments we use "#"

```
add x0, x1, x2
# This is ignored
sub x0, x1, x2.
```

BHATTI

CamScanner

y/date

Branches & Jumps:

Unlike mayn, we don't have cmp.

But like mayn we do have labels in exactly the same way.

Code:

```
        ---
        ---
        ---

Label 1:
        ---
        ---
        ---
        ---
        ---
        ---
```

Branches execute instructions out of sequence.

There are 2 types of branches:

BHATTI

CamScanner

day/date

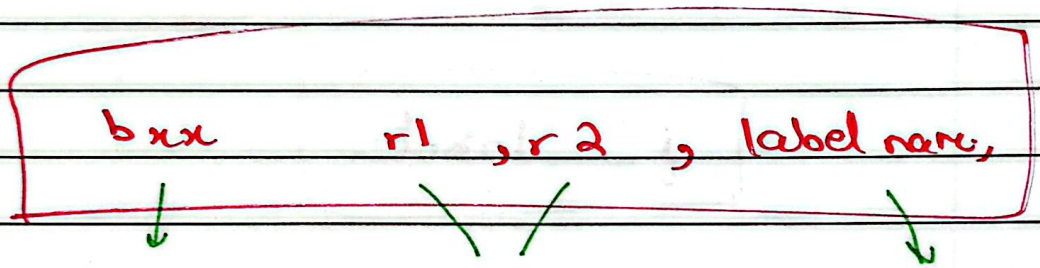
1) Conditional branches.

beq : branch if equal

bne : branch if not equal

bLt : branch if less than

bge : branch if greater than or equal.




```
if (r1 == r2) {        // if beq
    goto label;
}
```

BHATTI

CamScanner

day/date

2) Unconditional branches:

- j : jump
- jr : jump register
- jal : jump & link
- jalr : jump & link register.

we will learn these later.

```
j label
```

If this line runs ⇒ basically just go to the label.

BHATTI

CamScanner

day/date

Conditional statements:

These will be just like mars, ~~if else statements~~.

like we used to use jumps, here same way.

Example:

C code:

```
if (i == j)
    f = g + h
else
    f = f - i
```

RISC - V:

```
# s0 = f , s1 = g , s2 = h , s3 = i , s4 = j

beq s3 , s4 , Iflabel
sub s0 , s0 , s3 # else
j exit.
Iflabel:
add s0 , s1 , s2 // if-
exit:
```

if in paper values or memory of variables not given then at top just write a comment of what register represent what.

BHATTI

CamScanner

day/date

Loops: (while loops):

Again like mayam.

C Code:

```
int pow = 1
int x = 0;
while (pow != 128) {
    pow = pow * 2;
    x = x + 1;
}
```

CamScanner

day/date

RISC:

```
# s0 = pow , s1 = x.

xor  s0, s0 ,s0 # clearing out s0. s0 = 0.
addi s0, s0 ,1

add  s1 , zero , zero
addi s2 , zero , RB
whileLabel:
    beq s0 , s2 , exit

    slli s0 , s0 , 1 # shifting left one bit, multiplies by 2.

    addi s1, s1 , 1 ;

    j whileLabel;

Exit.
```

TIP:

Always add condition in start

If we add at end that means do while loop, 1 iteration would run no matter what.

BHATTI

CamScanner

For Loops

```
for (initialization; condition; loop operation)
  statement
```

- `initialization`: executes **before** the loop begins
- `condition`: is tested **at the beginning** of each iteration
- `loop operation`: executes at the **end** of each iteration
- `statement`: executes **each time** the condition is met

C Code

```
// add the numbers from 0 to 9
int sum = 0;
int i;
for (i=0; i!=10; i = i+1) {
sum = sum + i;
}
```

RISC-V assembly code

```
# s0 = i, s1 = sum
addi s1, zero, 0
add s0, zero, zero
addi t0, zero, 10
for:
    beq s0, t0, done
    add s1, s1, s0
    addi s0, s0, 1
    j for
done:
```

CamScanner

ay/date

Arrays:

In RISC-V array elements require pointer arithmetic using base address of array & element offsets.

Address of A[i] = Base address of A + (i * 4).

So first of load base address of an array into a register & then keep on adding 4 into it or i * 4 to traverse it.

We are considering for 32 bit (4B) words. (Not smaller (halfwords) or longer).

Example:

C code:

```
int array[1000];
int i;
for (i=0; i<1000; i=i+1)
    array[i] = array[i] * 8;
```

BHATTI

CamScanner

late

RISC-V.

s0 = array base address, s1 = i

adding base address & initialization.

```
lui  s0, 0x23B8F
ori  s0, s0, 0x400      } s0 = base address
```

```
addi s1, zero, 0
addi t2, zero, 1000

loop:
    bge  s1, t2, done
    slli t0, s1, 2      # t0 = i * 4
    add  t0, t0, s0     # address of array[i]
    lw   t1, 0(t0)      # load word into t1
    slli t1, t1, 3      # multiply by 8.
    sw   t1, 0(t0)      # save word back
    addi s1, s1, 1      # i = i+1
    j    loop           # repeat.

done:
```

BHATTI

CamScanner

day/date

SLT instruction:

set if less than.

slt x_1, x_2, x_3

if $x_2 < x_3$,
$x_1 = 1$
else
$x_1 = 0$.

can be used with beq / bge / blt.

BHATTI

CamScanner

day/date

Functions:

Functions are used to make a program more modular. but how to write them in RISC-V

In RISC-V we use labels to seperate functions e.g:

```
main:
    _ _
    ___
    ____
    ____

sum:

    addi x7 x0, x0, x5
```

- But we know how to go to sum label (use jump) but then how to come back?

  If we use even more labels to come back then what even is the point of modularity.

BHATTI

day/date

What we want is to save the return address of (PC+4) before jumping to function so we can return back there.

main :

PC → # sum is called here

(PC+4) → add ----

sum :

when sum is called its (PC+4) next instruction (as all instructions are 4 Bytes) should be saved.

BHATTI

CamScanner

day/date

So for this we have:

```
jal   funcname
```

jump & link.
now this register moves the (PC+4) / return address to x1 / ra, which is the return address register.

But then how to actually return from the called function:

```
jr    ra
```

jr — jump register.

How to pass argument?

we use (a0 — a7) registers.

BHATTI

CamScanner

day/date

How to get return value?

we use a0 register.

jr is actually a pseudo instruction made from jalr.

```
jr    ra
```

actually at backend.

```
jalr  zero 0(ra)
```

jalr = jump & link register.

```
jalr  rd , offset (rs1)
```

It jumps to offset + rs1 but also saves PC+4 into rd.

BHATTI

CamScanner

day/date

e.g.:

jalr x0, 0(ra)

x0 = PC+4

but x0 is not allowed to be changed.

This doesn't need to be changed as this is not error but actually the (PC+4) value is just lost

BHATTI

CamScanner

day/date

Examples:

C code:

```
int main ( ) {

    int y;

    y = diff_of_sums (2, 3, 4, 5);

}


int diff_of_sums (int a, int b, int c, int d) {

    int result;
    result = (a+b) - (c+d);

    return result;

}
```

BHATTI

CamScanner

day/date

RISC-V:

```
# s0 = y.
main:

    addi a0, zero, 2
    addi a1, zero, 3
    addi a2, zero, 4
    addi a3, zero, 5.

    jal  diff_of_sums

    add  s0, a0, zero.

# load back s0 to memory. & quit.
- - - - -

Diff-of-sums:

    add  t0, a0, a1
    add  t1, a2, a3
    sub  t0, t0, t1

    add  a0, t0, zero  # return value in a0.

    jr   ra.
```

filling argument registers.

BHATTI

CamScanner

day/date

Imp:

Now in previous code we can see that the function over write several registers that might have ~~to be used~~ values still required by the main function.

So we need to save value of register before hand.

So for that we use → Stack.

BHATTI

CamScanner

day/date

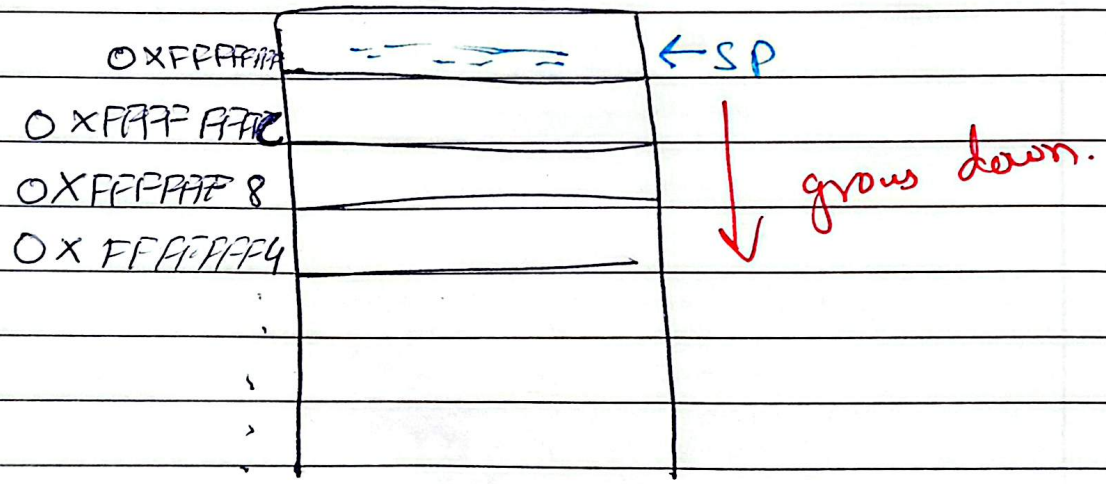
Stack of Functions:

Stack is the memory used to temporarily save variables.

It expands: when more space needed.

It contracts: when less space needed.

The stack grows down



"sp" registere (SC2) is used to Point/handle stack.

BHATTI

day/date

SP points to the top of stack

Remember in our last section of functions our diff_of_sums functions way over writing $t0 & $t1, we can save them in stack first & load them back at end.

* Remember stack is also memory so we use lw & sw to save there.

* But before saving we will need to make space on stack first ~~or~~ ~~over~~ ~~created~~ ~~space~~ like Push & Pop but push & pop are not in RISC-V, instead.

Push.
```
addi  sp, sp, -4.   # get 4B space down.
sw    t0, 0(sp)     # push t0.
```

BHATTI

day/date

Pop.
```
lw   t0, 0(sp)
addi sp, sp, 4.
```

The diff_of_sums example:

diff_of_sums:

```
addi sp, sp, -12
sw   s3, 8 (sp)
sw   t0, 4 (sp)
sw   t1, 0 (sp).
```
→ saving registers

```
add  t0, a0, a1
add  t1, a2, a3
sub  s3, t0, t1
add  a0, s3, zero
```

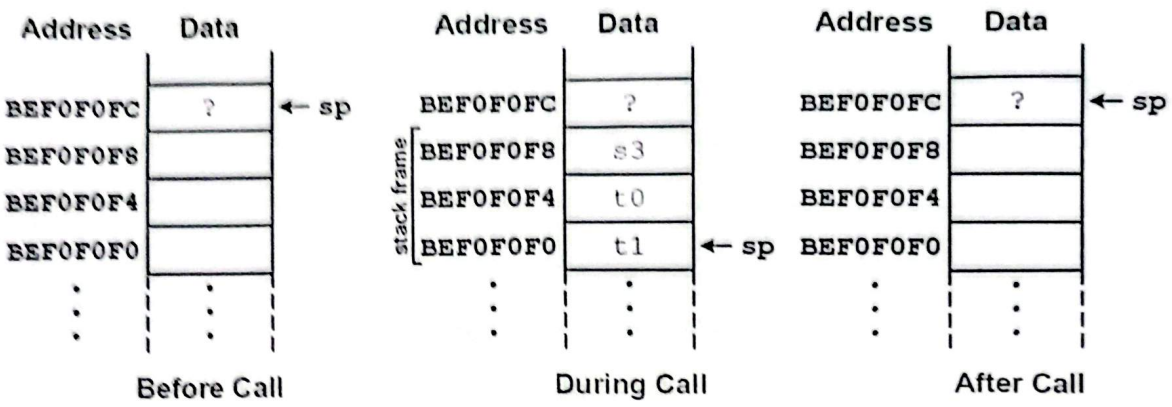
```
lw   s3, 8 (sp)
lw   t0, 4 (sp)
lw   t1, 0 (sp).
```
loading them back

```
addi sp, sp, 12
jr   ra
```

BHATTI

CamScanner

day/date



~~Done~~

~~done~~

TMP:

we typically don't need to ~~to~~ bring back a0, because in it we store the return value so no need to change it back to original

BHATTI

CamScanner

day/date

Non leaf Function calls:

Non leaf function:

A function (other than main) that calls another function (even it self).

Example:

```
main:
    ...
    jal f1
    ...

f1:
    ...
    jal f2
    ...
    jr ra

f2:
    ...
    jr ra.
```

f1 is a non-leaf function because it calls f2.

f2 is a leaf function.

BHATTI

CamScanner

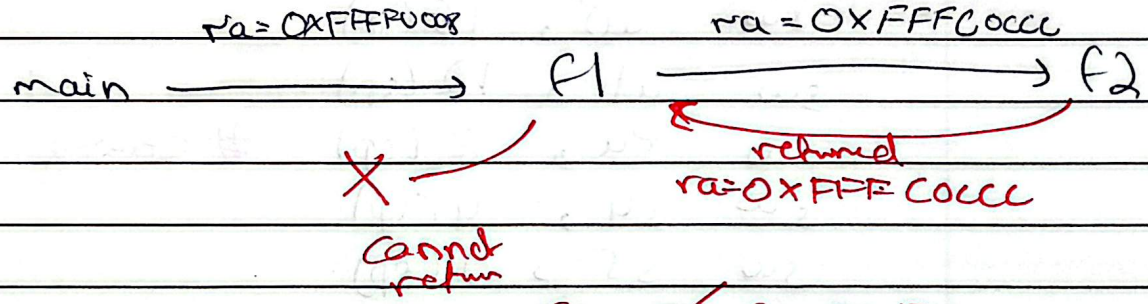
day/date

Now this introduces a problem:

- When main calls f1 consider (PC+4) value 0x FFFF0008.
  so
  ra = 0xFFFF0008. } f1 needs this to go to main back

- now when f1 calls f2, the ra register is over written by the new (PC+4) value.

So we lost our way back to main from f1.



BHATTI

CamScanner

day/date

To SOLVE this problem we must preserve our "ra" register value BEFORE a function call.
We do this by storing in Stack.

Example:

4 + ra = 5 total variables saving.

f1 (non-leaf) : uses $s4 & $s5 & needs a0, a1 after # f2

```
f1:
    addi  sp, sp, -20      # 5 words needs 5x4=20 Bytes.
    sw    a0, 16(sp)
    sw    a1, 12(sp)
    sw    ra, 8(sp)        # saving ra
    sw    s4, 4(sp)
    sw    s5, 0(sp).

    jal   f2

    lw    ra, 8(sp)
    # Bring back other registers as used.
    addi  sp, sp, 20       # fix sp position.
    jr    ra.
```

BHATTI

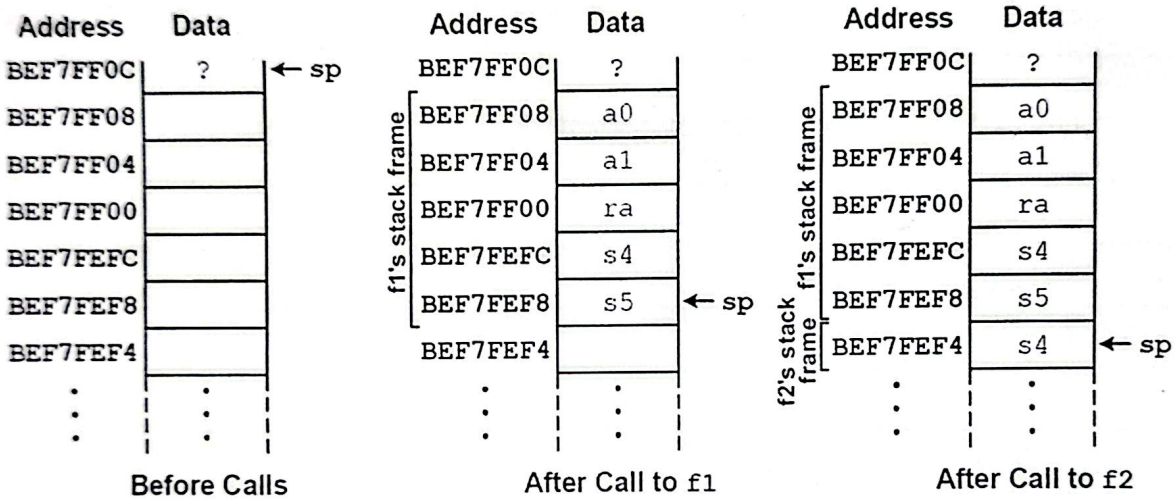
CamScanner

day/date

f2:

```
addi  sp, sp, -4
sw    s4, 0(sp)
 - -

lw   s4, 0(sp)
addi sp, sp, 4
jr   ra
```



BHATTI

day/date

Recursive Function:

A function that calls itself.

In this always remember:

- Save ra, except if its the base case (in that it is a leaf function).

- Save argument or variable a function Before calls: Every time calls itself, it might need new argument like (n-1) or might change so do save them before hand.
  e.g a function might change (a0) to a return a value but we had a value stored in a0 so it is lost.

BHATTI

CamScanner

day/date

- Recursive call should come first

e.g

return (factorial (n-1) * n)

When the statement of recursion call comes. save values we might need. then change argument then do the recursion call.

After that when we return from recursion call do "* n"

e.g

return fibonacci (n-1) + fibnacci (n-2)

save ao

(1) ao = n-1

step.

(2) do jal fibonacci

(3) Bring back original ao.
load ao # still do not remove from stack.

(4) ao = n-2.

(5) jal fibonacci.

(6) Bring ao back again.

BHATTI

CamScanner

day/date

Example:

C code:

```
int factorial (int n) {
    if (n <=1)
        return 1;
    else
        return (n* factorial (n-1)).
```

BHATTI

CamScanner

day/date

RISC-V:

```
factorial:  # it has n in a0.

addi  sp, sp, 8
sw    a0, 4(sp)
sw    ra, 0(sp).

addi  t0, zero, 1

bgt   a0, t0, else   # if n>1 goto else.
addi  a0, zero, 1
addi  sp, sp, 8.
jr    ra.

else:
addi  a0, a0, -1
jal   factorial
lw    t1, 4(sp)
lw    ra, 0(sp)
addi  sp, sp, 8
mul   a0, t1, a0
jr ra.
```

base case
so, a0=1
return.
no need to multiply anything.

recursive call first

bring a0 to t1.

multiply a0 * t1

a0 brought back from recursion.

t1 is stored in a0

saved back in a0, as that is our return value storage.

BHATTI

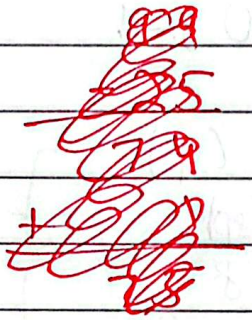
day/date

Important learnings about some assembly instructions:

- Negative number:

Whenever we use a negative number like store it or save it is like addi at backend, 2's complement happens & that is how it is stored. so do 2's complement after converting to them.

e.g: addi s0, zero, -25



25 = 19 (Hex).

only converts number not sign.

-ve says take 2's comp of the +ve constant

FFFFFFFF

- 00000019

FFFFFFE6

+ 1

-25 = FFFFFFE7.

BHATTI

day/date

another.

addi $0, $zero, -1

-ve so it says take 2's complement of 1:

FFFFFFFF

− 00000001

FFFFFFFE

+ 1

FFFFFFFF

2's complement subtract from FFF.. & then add one.

Now when these numbers are stored like this in signed form. But if an instruction like divu (division unsigned) comes then these become huge numbers because all bits are considered to make constant, none are for sign.

FFFFFFFF (unsigned) = 4295,967,296.

FFFFFFFF (signed) = −1.

BHATTI

CamScanner

day/date

- converting a signed number:

e.g what does FFFF 3567 mean? in signed:

Take 2's complement:

FF FF BFFF
FFFF 3567
0000 C698
+ 1
0000 C699

- 2's complement
- invert bits
- add 1.

0000 C699 = 51865

so **-51865**

- So for converting: back to decimals
  - use the positive constant
  - take 2's complement
  - add a negative sign

BHATTI

day/date

- division & remainder of unsigned:

Unsigned division & remainders have their own commands as well:

```
divu   x1, x2, x3.
```

```
remu   x1, x2, x3.
```

* Remember: The remainder always gets the sign of the dividend.

If dividend is +ve, remainder is +ve.

if divident -ve, remainder is negative.

* division get sign according to normal math eg $\frac{-1}{-1} = +1$, $\frac{-1}{+1} = -1$,

$\frac{+1}{-1} = -1$, $\frac{+1}{+1} = +1$

BHATTI

CamScanner

date

examples:

1)

```
addi  s0, zero, -25  # S0 = FFFFFFE7
addi  s1, zero, 2    # S1 = 00000002.
```

(a) `div  s2, s0, s1` # S2 = -12

(b) `divu  s3, s0, s1` # S3 = 7FFFFFF3.

(c) `rem  s4, s0, s1` # S4 = FFFFFFFE

(d) `remu  s5, s0, s1` # S5 = 00000001

(a) signed division

$$\frac{-25}{2} = -12$$

(b) unsigned division.

(unsigned so now all bits considered as magnitude bits)

$$\frac{FFFFFFE7}{00000002} = \frac{4294967271}{2}$$

$$= 7FFFFFF3$$

BHATTI

lay/date

(c) signed remainder:

$-25 \% 2 = -1$

sign of dividend.

(d) unsigned remainder:

$4294967271 \% 2 = +1$

now both are positive

day/date

MUL, mulh, mulhu, mulhsu.

mul x1, x2, x3. — signed signed. $x_1 = x_2 * x_3$ lower 32 bits

mulh x1, x2, x3 — signed signed. $x_1 = x_2 * x_3$ upper 32 bits

mulhu x1, x2, x3 — upper 32 bits. unsigned unsigned. $x_1 = x_2 * x_3$

mulhsu x1, x2, x3

$x_1 = x_2 * x_3$.

upper 32 bits. / signed / unsigned.

e.g consider:

addi S0, zero, 4 # S0 = 4 = 00000004

addi S1, zero, -5 # S1 = -5 = FFFFFFFB

= 4294967292

① mulhu S3, S0, S1

② mulhsu S3, S0, S1

③ mulhsu S3, S1, S0

BHATTI

CamScanner

day/date

① multu $s3, $s0, $s1

unsigned multiplication so:

= 4 * 4294967292

= 0x0000000003 _ FFFFFFEC

upper

so $s3 = 3.

② multsu $s3, $s0, $s1

considered signed — unsigned:

= 4 * 4294967292, $s3 = 3.

③ multsu $s3, $s0, $s1:

= -5 * 4 = -20.

now considered signed.

upper bits

$s3 = FFFFFFFF

BHATTI

CamScanner

day/date

Align & data Insertion in memory:

For data Insertion we have directives in RISC-V.

label	Directive	values (comma seperated)

e.g

b1 : .byte 0xAD

• byte:

range : [-128, 255]

b1 : .byte 0b101010, 0x42, 255 ✓ allowed.

b2 : .byte -129 ✗ not allowed error.

b3 : .byte "h." ✗ not allowed, " " add a zero at end, for error.

b4 : .byte 'a' ✓ allowed

BHATTI

CamScanner

day/date

- string: / .asciz:

both are exactly same.

example.

b1 : .string "hello." ✓ work

b2 : .string 'hello' ✗ error double inverted commas are necessary.

- half:

This is for half words (2Bytes).
half words are required to be aligned on half boundary (2B) by
-align 2:

range: [-32768 , 65536]

BHATTI

day/date

- word:

(need to aligned on word boundary by - align to 4.

range : [-2147483648, 4294967295]

Align directive:

.align n

It align memory on 2^n.

eg

align 2

$= 2^2 = 4$

so in multiples of 4

If multiple of 4, address start from next multiple of 4

eg 0, 4, 8, 12(8)

BHATTI

CamScanner

date

If

align 4

$2^4 = 16$.

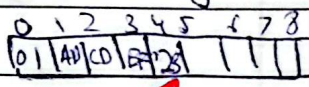
so

multiples of 16.

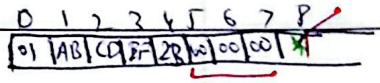
next address starts from 0.

example. (align 2)

consider memory currently as



now if we do align 2.



aligned till we reached a multiple of 4, next address starts from 8

More about memory.

- Little endian is followed.

BHATTI

CamScanner

y/date

Loading Instructions : More variation:

1) la

la register , datalabel

It loads the address of ~~data~~ data when the data resides in to the register.

la = load address.

.data

b1 : .Byte 0x77 #smaller on address 0x10000004

label

.text

main:

la x5 , b1

x5 = 0x10000004

x5 now has address on which b1 starts.

BHATTI

CamScanner

day/date

2) lb
load byte

lb xregister, offset (base address register)

But our registers are only 32-bit. How can we load a byte into it??
sign extension.

lb takes the byte, sign extends to 32 bit & places in register.

remember the register is emptied out of its content & all new data is written in all 32 bits.

e.g.:

consider x10 has address pointing to data 0x24.

BHATTI

CamScanner

lay/date

LB , x5 , 0 (x10)

#x5 = 0x00000024.

#consider x10 is pointing to data 0x8F

Lb x5 , 0 (x10)

sign extended.

x5 = 0xFFFFFF8F

3) Lbu , Lh , Lhu.

Lbu = load byte unsigned (zero-extension)

Doesn't signe extend but zero extnd.

Lh = load halfword signed (sign-extnsn)

This will now load 2B.

(keep little endian in mind while loading)

sign-extended.

BHATTI

CamScanner

/date

lhu = load half word unsigned, (zero-extension)

load 2 Byte but it will zero-extend.

4) Li : load instruction.

This is basically a pseudo instruction that does the work of adding a 32 bit immediate into a register

It uses the same concepts of lui & addi at backend

Li register, 32-bit immediate

e.g

li s0, 0x1023 7856

BHATTI

day/date

Saving Instructions more variations:

1) sb : save byte

`sb register , offset (base address)`

It takes the last 8 bits from the register & writes into 1B of memory.

It doesn't change the next 3 B as well. It only changes the byte it is pointing to.

2) sh: save half word:

`sh register , offset (base address)`

It takes last 16 bits (2B) from register & stores / changes exactly 2B where its pointing in memory.

BHATTI

CamScanner

te

Pseudoinstruction: bgt & ble:

we have conditionals ~~blt~~ & blt & bge as recall RISC-V commands but

consider

b > c

we can write this as

b >= (c-1)

& use bge

but this uses an extra instruction.

So what to do:

Simply just flip the statement

b > c ≡ c < b

how we can use

blt c, b, label.

BHATTI

late

This is exactly what pseudo instructions bgt & ble do:

bgt x_1, x_2, label $\equiv$ blt x_2, x_1, label

(inverted)

ble x_1, x_2, label $\equiv$ bgt x_2, x_1, label

So we can use bgt & ble as well.

BHATTI

CamScanner

More Pseudo Instructions:

Pseudoinstruction	RISC-V Instructions
`j label`	`jal zero, label`
`jr ra`	`jalr zero, ra, 0`
`mv t5, s3`	`addi t5, s3, 0`
`not s7, t2`	`xori s7, t2, -1`
`nop`	`addi zero, zero, 0`
`li s8, 0x56789DEF`	`lui s8, 0x5678A` `addi s8, s8, 0xDEF`
`bgt s1, t3, L3`	`blt t3, s1, L3`
`bgez t2, L7`	`bge t2, zero, L7`
`call L1`	`auipc ra,` $imm_{31:12}$ `jalr ra, ra,` $imm_{11:0}$
`ret`	`jalr zero, ra, 0`

CamScanner